

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

1INF29 SISTEMAS OPERATIVOS

Semestre 2025-1

Laboratorio 2

1) (10 puntos) A ChatGPT (<https://chatgpt.com/>) se le envió el siguiente requerimiento:

El siguiente código genera un abanico de procesos.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

/* Este programa crea un abanico de procsos. */
/* Ejm 2.6 del libro UNIX Programacion Practica - Kay Robbins */
/* Steve Robbins */

#define N 8

int main(void)
{ int i,status;
  pid_t child,pid_padre;

  pid_padre=getpid();
  for(i=0;i<N; ++i)
    if((child=fork())<=0) break;
  if(pid_padre==getpid()) for(i=0;i<N;++i) wait(&status);
  return 0;
}
```

Modifique el código para que el primer proceso creado por el padre, elimine a todos los procesos hermanos, y también al proceso padre, enviando la señal SIGKILL.

ChatGPT respondió con el siguiente código (se encuentra en Paideia como *l.c*)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <sys/types.h>
6 #include <signal.h>
7
8 #define N 8
9
10 int main(void) {
11     pid_t pid_padre = getpid();
12     pid_t pids[N]; // Almacenar los PIDs de los hijos
13     pid_t child;
14     int i, status;
15
16     for (i = 0; i < N; ++i) {
17         child = fork();
18         if (child < 0) {
19             perror("fork");
20             exit(1);
21         }
22
23         if (child == 0) {
24             // Proceso hijo
25             if (i == 0) {
26                 // Primer hijo
27                 // Esperamos un poco para asegurarnos que los demás hijos se creen
28                 sleep(1);
29
30                 // Eliminamos a los demás hijos
31                 for (int j = 1; j < N; ++j) {
32                     kill(pids[j], SIGKILL);
33                 }
34
35                 // Eliminamos al padre
36                 kill(pid_padre, SIGKILL);
37
38                 printf("Primer hijo (%d): Eliminé a mis hermanos y al padre.\n", getpid());
39             }
40
41             // Todos los hijos esperan indefinidamente
42             pause();
43             exit(0);
44         } else {
45             // Proceso padre guarda el PID del hijo
46             pids[i] = child;
47         }
48     }
49
50     // Solo el padre hace wait si no ha sido eliminado
51     if (getpid() == pid_padre) {
52         for (i = 0; i < N; ++i) {
53             wait(&status);
54         }
55         printf("Padre ha terminado.\n");
56     }
57
58     return 0;
59 }
```

a) (4 puntos) En un archivo texto con nombre *IA.txt* explique porqué este código es incorrecto, es decir no cumple con lo especificado en el problema. Copie el archivo *I.c* con nombre *IA.c* y añada al código líneas que demuestren que el código es incorrecto. Emplee las salidas del código modificado para justificar su explicación. Debe presentar el código modificado *IA.c* y el archivo texto *IA.txt*.

b) (2 puntos) En un archivo con nombre *IB.txt* explique por qué aparentemente el programa se comporta como si verdaderamente cumpliera lo especificado.

c) (4 puntos) Copie el archivo *I.c* con nombre *IC.c* y modifíquelo de forma que resuelva el problema solicitado.

2) (10 puntos) Se tiene un proceso principal que crea 2 procesos hijos. Cada uno de los procesos hijos genera 10 números aleatorios entre 1 y 500. Se desea que el primer proceso imprima todos los números impares generados por ambos procesos hijos. Y el segundo imprima todos los pares, generados por ambos procesos hijos. La impresión debe tener el siguiente formato:

<pid del proceso que imprime> <número impar/par> <pid del proceso que lo generó>

En Paideia se le proporciona el código base: *2.c*

Porf. Alejandro T. Bello Ruiz